Sketch2Shape AI Project Report

Technical implementation, user guide, verification results and submission readiness

TechZephyr 4.0 | IIT Bhubaneswar | 31 October 2026

Overall conclusion

The supplied project is a working local sketch-to-volume prototype with an integrated but unverified generative AI path. It does not yet demonstrate every requirement of the hackathon brief. The CPU workflow has been executed successfully, including geometry generation, interactive preview, GLB and OBJ export, and automated checks. ControlNet and TripoSR integration code is present, but their weights and runtime have not been installed and executed on this machine.

Before claiming full compliance, the team must complete a successful AI run, inspect the generated asset, record the actual AI hardware and runtime, and reproduce that run from a clean environment. The included sample is explicitly a CPU result. It must not be described as AI-generated.

Purpose of this report

This report explains what the code package contains, how each part works, why the technologies were selected, how to install and use the application, what was actually tested, and what remains for a credible submission. It is intended for the project team and evaluators. It describes the delivered implementation rather than assuming that every proposed capability has already been demonstrated.

Area	Current evidence
Input and interface	Drawing and upload implemented; sample-to-model browser workflow exercised.
CPU reconstruction	Fresh sketches produce distinct, closed volumetric meshes.
AI reconstruction	ControlNet refinement and TripoSR adapter implemented; no verified inference run.
Deliverables	Source, setup guide, tests, model exports, input samples and run report included.
Submission readiness	Partial. AI execution evidence and a clean AI environment reproduction are outstanding.

How to read this report

Sections 1 to 3 describe scope and requirement coverage. Sections 4 to 6 explain the implementation. Sections 7 and 8 provide installation and operating instructions. Sections 9 to 11 cover evidence, limitations and the final submission steps.

1 Problem and project scope

The challenge is to convert a single 2D sketch into a usable 3D asset. A sketch supplies contours and visual intent but does not uniquely specify depth, hidden surfaces, material, scale or topology. A working system therefore needs both input processing and a method for interpreting or approximating the missing information.

Required outcome

The application should accept a drawn or uploaded sketch, allow an optional descriptive prompt, produce recognizable three-dimensional geometry, provide suitable surface appearance, and export at least one supported asset format. The team must supply source code, reproducible setup instructions, an output artifact, example results, resource assumptions and a live end-to-end demonstration.

Implementation approach

The project provides two selectable workflows. CPU mode constructs a rounded volume directly from the sketch silhouette. It provides a functional baseline for input handling, geometry creation, preview and exports without neural weights. AI mode first refines the sketch into an object image using Stable Diffusion with scribble ControlNet, then sends that image to the official TripoSR program for 3D reconstruction and surface color estimation.

These workflows have different evidentiary value. The CPU workflow proves that the application generates new 3D geometry from new input. It does not demonstrate generative AI, semantic object understanding or material interpretation. The AI workflow is the intended route for meeting those portions of the brief, but integration code alone is not proof of successful execution.

What is intentionally outside scope

The system does not promise engineering accuracy, manufacturing scale, printability, animation-ready topology or exact recovery of hidden surfaces. A public website is not required and none was deployed. The delivered application runs locally in a browser and also supports a command-line interface.

Development work completed

The work produced the browser interface, local API, job queue, preprocessing, distance-field reconstruction, export validation, AI stage adapters, CLI, automated tests, bundled viewer, sample files, setup documentation and demonstration checklist. No neural network was trained, no custom dataset was collected, and no model fine-tuning or accuracy benchmark was performed.

2 Requirements assessment

Status terms: Verified means exercised locally; Implemented means present in source but not fully exercised in the relevant configuration; Outstanding means further execution or evidence is required. This is a qualitative assessment, not a predicted judging score.

Requirement	Status	Evidence or remaining work
In-app drawing	Verified in part	Brush, eraser, undo and reset exist; sample input and generation exercised in browser.
Image upload	Verified	API accepts decoded sketch images; invalid-image handling tested.
Optional prompt	Partial	Field and AI default prompt exist; CPU works with no text; AI optional-text run unverified.
Input preprocessing	Verified	Cropping, normalization, alpha handling and mask preparation executed.
Computer vision	Verified	Thresholding, morphology, contour-derived mask and distance field executed.
Generative AI use	Outstanding evidence	ControlNet code is present; install weights and complete an inference run.
Sketch fidelity	Partial	CPU volume follows the silhouette; AI fidelity has not been assessed.
Actual 3D geometry	Verified for CPU	Fresh outputs have triangles and positive extents on all three axes.
Appearance	Verified for CPU	GLB preserves vertex color; AI surface estimation unverified.
GLB or OBJ export	Verified	Both formats exported and reopened through trimesh.
Interactive preview	Verified for CPU	Browser displayed generated geometry with orbit controls.
Mesh validation	Verified	Checks finite vertices, faces and extent; reports watertightness.
Fresh judge sketch	Partial	Fresh CPU inputs tested; fresh AI input still needs a live demonstration.
Source and README	Included	Code, dependencies, architecture and usage instructions provided.
Output artifact	Partial	CPU input and GLB/OBJ included; add an AI-generated sample.
Runtime and resources	Partial	CPU measurements supplied; AI runtime and VRAM remain unmeasured.
Reproducible installation	Partial	App versions listed; clean setup and upstream AI lock still need verification.
Model disclosure	Included	Model IDs, repository, library roles and license references documented.

3 Package contents and responsibilities

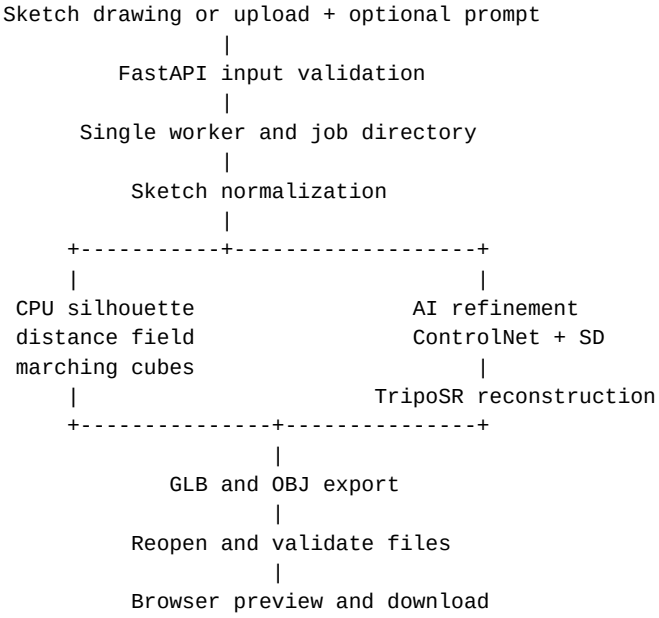
The downloadable archive contains a top-level Sketch2Shape folder. Extract that folder before running commands. Keep the application files together because the server serves its own static assets and resolves its AI helper relative to the project directory.

File or folder	Purpose
app/static/index.html	Application layout, drawing canvas, settings, preview and downloads.
app/static/style.css	Responsive visual styling for desktop and smaller screens.
app/static/app.js	Pointer drawing, upload, undo history, job submission, polling and result links.
app/static/vendor	Bundled model-viewer JavaScript and its license; no viewer CDN required.
app/server.py	FastAPI application, upload validation, queue, job status and result endpoints.
app/pipeline.py	Shared preprocessing, CPU reconstruction, AI invocation, export and validation.
app/refine.py	ControlNet image refinement; runs in the separate AI environment.
app/cli.py	Command-line entry point using the same generation pipeline.
requirements.txt	Pinned application package versions matching the tested host packages.
requirements-ai.txt	Pinned packages for the image generation stage; not a full TripoSR lock.
start.ps1	Windows environment creation and local server launcher.
tests	Geometry and API integration tests; test dependency list.
examples	Input sketch and verified CPU GLB/OBJ, intermediate images and run metadata.
docs	Requirement mapping, demonstration instructions, verification and this report.
README.md and LICENSE	Setup, usage, limitations, attribution and original-code license.

Generated data

Normal app usage creates runs under runs/<job-id>. That folder is runtime data and is excluded from the clean source ZIP. Every successful job records the input, intermediate images, models and report.json. The Download Run bundle button packages that particular job for inspection or submission.

4 Architecture and technology choices



Technology	Why it is used
Python and FastAPI	Connect image processing and model adapters through a small local HTTP service.
Pillow and NumPy	Decode images, handle transparency/orientation and operate on image arrays.
SciPy	Morphology, distance transforms and scalar-field smoothing for the CPU baseline.
scikit-image	Marching cubes converts a sampled 3D field into a triangle surface.
trimesh	Construct meshes, repair normals, export assets and reopen files for validation.
HTML CSS JavaScript	Implement drawing, settings, asynchronous progress and download controls.
Google model-viewer	Display self-contained GLB assets with interactive orbit and zoom.
Diffusers and ControlNet	Condition visual generation on the sketch as well as optional text.
TripoSR	Use a pretrained image-to-3D model instead of training a reconstruction network.

The separation between interface, shared pipeline and AI subprocesses keeps the CPU app usable while allowing a dedicated model environment. The AI stages run sequentially in separate processes, so their model allocations can be released between stages. This reduces simultaneous memory demand but does not establish a measured VRAM requirement.

5 Image processing and CPU reconstruction

Input preparation

The server decodes PNG, JPEG or WebP input rather than trusting a filename alone. It enforces a 10 MB upload limit and a 20 megapixel image limit. The pipeline applies EXIF orientation, composites transparent pixels onto white, and limits image dimensions. Border brightness is used to detect a dark background and invert it when appropriate.

Pixels below grayscale value 190 are treated as sketch strokes. The content bounding box is cropped with a small margin, scaled to fit within 420 by 420 pixels, and centered on a 512 by 512 white image. CPU mask processing works at 192 by 192. Small gaps are closed, enclosed regions are filled, and a small dilation gives thin content some support. Tiny disconnected components are removed relative to the dominant region.

How volume is constructed

A distance transform measures how far each foreground pixel lies from the silhouette boundary. Points farther inside the silhouette receive a larger thickness. The local thickness is proportional to the square root of normalized distance and the user-selected depth. The algorithm samples 64 positions along the depth axis and constructs a signed field whose zero level defines the object surface.

```
d = distance to silhouette boundary
r = sqrt(d / max(d)) * image_size * depth / 2
field(x, y, z) = r(x, y) - abs(z)
```

Outside the silhouette the field is negative. Gaussian smoothing softens transitions, and marching cubes extracts the zero-level surface as triangles. Vertices are reoriented, centered and scaled so the longest bounding-box dimension is two arbitrary units. Mesh normals are repaired and the selected color is assigned to vertices.

What the result means

This produces a rounded three-dimensional solid, not a screenshot or flat textured plane. Its depth and closed surface are measurable. However, the thickness is a geometric assumption: the algorithm does not understand object categories, interiors or perspective. A vase sketch becomes an inflated vase-shaped solid; it is not guaranteed to be a hollow, rotationally symmetric vessel. Open strokes can become tubes, while hole filling may remove intended openings.

6 Generative AI integration and export

Sketch guided image generation

AI mode launches app/refine.py using AI_PYTHON. The helper uses StableDiffusionControlNetPipeline with stable-diffusion-v1-5/stable-diffusion-v1-5 and lllyasviel/control_v11p_sd15_scribble. Dark sketch strokes are converted to white strokes on a black control image. The sketch therefore conditions generation on every AI request, including requests with no user text.

The optional description is combined with a default single-object, centered product-image description. The current configuration uses 30 inference steps, guidance scale 7.5 and ControlNet conditioning scale 1.15. A seed controls initial noise. CUDA uses half precision and CPU offload; CPU inference uses float32. The helper retains the pipeline safety check and reports filtered output as a failure. These settings are implemented, not benchmarked here.

Image to 3D reconstruction

The generated refined.png is passed to the official TripoSR run.py in TRIPOSR_DIR. The adapter requests GLB output, marching-cubes resolution 256 and the configured device. The upstream default foreground removal is retained. The app expects triposr/0/mesh.glb and treats missing output or a failing process as an error. Each AI stage writes a separate log and has a 30-minute timeout.

TripoSR weights are obtained through its standard stabilityai/TripoSR model source. The project does not redistribute those weights. The AI environment also depends on upstream native components, including torchmcubes, and needs a compatible PyTorch and CUDA installation. Merely setting the environment paths does not verify these dependencies.

Export and validation

GLB is the preferred asset because it can retain the reconstructed surface appearance in one file. The CPU result uses a uniform vertex color; the AI route is intended to preserve TripoSR vertex colors. This is not UV texture baking, and no image texture atlas is generated by this implementation. OBJ is a secondary geometry-only export.

After export, both files are reopened with trimesh. Validation rejects empty triangle sets, non-finite vertex coordinates and near-zero extent on any axis. It records vertex count, triangle count, extents, visual representation and watertightness. These checks do not measure semantic fidelity, self-intersections, manufacturability, UV quality or independent-editor compatibility.

7 Installation and configuration

Run the basic app on Windows

Install Python 3.11 or newer for the application, extract the ZIP, and open PowerShell in the extracted Sketch2Shape folder. Python 3.13.2 was used for local verification. Run the following commands and then visit http://127.0.0.1:8000 in your browser.

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m uvicorn app.server:app --host 127.0.0.1 --port 8000
```

Enter each command as a single line, or use the ready-made ./start.ps1 launcher. If script execution is restricted, use the explicit Python commands. Stop the server with Ctrl+C. Keep one server worker; the queue is not designed for multiple worker processes.

Linux or macOS equivalent

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
.venv/bin/python -m uvicorn app.server:app --port 8000
```

Configure AI mode

Create a separate Python 3.10 or 3.11 environment, install a suitable PyTorch build, clone the official TripoSR repository, and install its requirements plus requirements-ai.txt. Detailed commands are in README.md. A Linux machine with a compatible NVIDIA GPU is the preferred deployment target for the native AI dependencies. This environment has not been installed or validated here.

```
# Windows configuration after installing the AI environment
$env:TRIPOSR_DIR = (Resolve-Path vendor/TripoSR).Path
$env:AI_PYTHON = (Resolve-Path .ai-venv/Scripts/python.exe).Path
# Start the local server in this same terminal session
```

The app enables AI mode when AI_PYTHON exists and TRIPOSR_DIR contains run.py. The first AI request can download several model files and may need Hugging Face access approval or authentication. Follow the model terms; do not include tokens in a submission. Once dependencies and weights are cached, local inference does not require a paid API.

Resource planning

The upstream TripoSR README states approximately 6 GB VRAM for default image inference settings. Allowing 8 to 12 GB VRAM and 16 GB RAM for the combined workflow is a planning estimate, not a measured requirement. Record actual peak memory and elapsed time on the final evaluation machine. Model downloads and native builds also require disk space and network access.

8 How to use the project

Create and generate

1. Start the app and open the local address. Draw a dark closed outline on the light canvas, choose Upload sketch for an existing image, or select Try a sample vase. Use the brush size control, eraser, undo or clear to revise the concept. For predictable CPU results, use one prominent closed silhouette with a clean background.

2. Select CPU mode for the verified local geometric workflow. Choose the surface color and depth. Text is optional and is not semantically interpreted in CPU mode. Select AI only after completing the model setup. In AI mode, describe color and material in the optional prompt; the CPU color and depth settings do not guide the neural reconstruction.

3. Select Bring it to shape. The app submits the canvas image and settings, then polls the job status. Stages include preparation, optional AI refinement, reconstruction, validation and completion. The app processes one generation at a time and rejects additional submissions once three active jobs are present.

Inspect and export

4. When generation succeeds, drag the model to orbit and scroll to zoom. Inspect from the side and rear to confirm there is meaningful depth. The interface shows triangle count, measured pipeline time and a geometry status. Intermediate images help explain how the input was processed.

5. Choose Download GLB for the appearance-preserving asset. Choose OBJ if a geometry-only format is preferred. Download Run bundle to keep input, intermediates, exports, logs and metadata together. Open the GLB in an independent editor such as Blender before the final submission; that editor check has not been performed here.

Command line alternative

```
.\.venv\Scripts\python.exe -m app.cli examples/vase-sketch.png
    --output runs/my-vase --color "#77b8ee"
```

Enter the command on one line. Use --mode ai and --prompt to select the AI route after configuration. The output directory must be new or empty. The CLI prints progress and the same final report used by the web application.

Run report fields

report.json records the selected mode, prompt, seed, color, depth, timings, geometry validation, input SHA256 hash, operating system, processor, logical CPU count and model identifiers. The current report does not automatically capture GPU name, GPU peak memory, every package version or the TripoSR commit. Add those separately for the final AI submission.

9 Verification and measured results

All measurements below are local CPU results. They must not be used to predict AI generation time or quality. The reference sample is examples/vase-sketch.png and its outputs are in examples/cpu-vase.

Measurement	Observed result
Operating system	Windows 11, build 10.0.26300
Processor	Intel64 Family 6 Model 142 Stepping 12; 8 logical CPUs
Python and PyTorch	Python 3.13.2; PyTorch 2.12.1+cpu; CUDA unavailable
Preprocessing time	0.101 seconds
Reconstruction time	0.563 seconds
Total pipeline time	1.685 seconds, excluding interpreter startup
Vertices and triangles	43,834 vertices and 87,664 triangles
3D extents	1.337 x 2.000 x 1.098 arbitrary units
Surface status	Watertight; per-vertex color retained after GLB reopening
Browser example	97,744 triangles in 2.4 seconds; colored model displayed

Automated tests

Three tests passed in 7.357 seconds. The geometry test generated two fresh, different sketches and confirmed different output assets, more than 100 triangles, positive 3D extents, watertight surfaces, vertex appearance and a readable OBJ. The input test rejected an empty sketch and accepted transparent image content. The API test checked invalid-image rejection, unknown job handling, upload, asynchronous completion, GLB header, ZIP contents and delivery of static application assets.

```
.\.venv\Scripts\python.exe -m pip install -r tests/requirements.txt
.\.venv\Scripts\python.exe -m unittest discover -s tests -v
```

Evidence limits

The tests used existing host packages plus workspace-installed dependencies, not a newly created environment installed entirely from the README. Browser verification confirmed the sample-to-preview flow. It did not exhaustively test every drawing tool, mobile gesture, queue interaction or external 3D editor. AI weights, native compilation, inference, actual GPU runtime and perceptual output quality remain untested.

10 Limitations and troubleshooting

Issue	Cause and practical action
AI option disabled	The configured paths are absent or invalid. Set both environment variables in the server session and restart.
AI process fails	Read controlnet.log or triposr.log in the run bundle. Check imports, native build, model access and device compatibility.
GPU out of memory	Close competing GPU jobs; consider lower reconstruction resolution. Measure the quality tradeoff.
Poor CPU geometry	Use a larger closed outline. The baseline has no category understanding and can merge strokes or fill openings.
Empty sketch error	Increase content size or contrast. Dark marks on a clean white background are easiest to process.
Preview fails	Ensure bundled viewer files exist and WebGL is available. Inspect the exported GLB in another editor.
Job interrupted	A server restart ends the active process. Restart generation; unfinished persisted jobs are reported as failed.
Queue full	Wait for an active job to finish. The local queue allows three active jobs including the running one.
OBJ has no colors	This export intentionally contains geometry only. Use GLB to preserve surface appearance.

Quality limitations

A single view cannot uniquely determine hidden geometry. Even after AI setup, consistency with a sketch is an objective to evaluate, not a guarantee. Thin structures, overlapping lines, multiple objects and unusual viewpoints are likely difficult cases. Vertex colors provide visible appearance but are different from detailed UV-mapped textures and physically based materials.

Operational limitations

The server is for local use and binds to 127.0.0.1. It has no login, per-user storage isolation or production deployment configuration. Jobs and sketches remain on local disk until manually removed. There is no cancellation interface or automatic storage cleanup. Each AI stage has a timeout, but there is no calibrated completion-time estimate.

The application pins its main dependencies, while the upstream TripoSR installation is not locked to a tested commit. The AI setup may require compatibility fixes. The current code detects existing paths rather than conducting a full model readiness test. These are material reproducibility limitations for judging and must be disclosed.

11 Submission completion and references

Required work before claiming full compliance

1. Prepare the AI environment on a compatible machine. Verify imports, CUDA, torchmcubes, model access and pip check. Record the Python version, operating system, GPU, VRAM, RAM, exact TripoSR commit and installed packages.

2. Run an AI generation from a fresh sketch with no user text. Then run another with a material or color prompt. Confirm that sketch structure influences the result. Keep the actual inputs, refined images, GLB files and complete run bundles.

3. Inspect the output in the embedded viewer and an independent 3D editor. Verify nonempty geometry, depth and surface appearance. Assess recognizability and artifacts honestly; numerical mesh validity alone cannot establish visual quality.

4. Measure total elapsed generation time on the final hardware, distinguish first-time downloads from warm execution, and record peak memory if available. Include failures or constraints encountered during evaluation.

5. Reinstall from a clean environment using the final documented versions and repeat generation. Capture a dependency lock for the working AI setup and revise the README if any undocumented steps were needed.

6. Add the verified AI samples and environment evidence to the submission. Replace the outstanding status entries only when the relevant actions have succeeded. Include team identity and any event-specific submission details separately; none were invented in this package.

Suggested live demonstration

Start the app, explain the pipeline, accept a fresh judge-provided sketch, and run AI mode. Show the intermediate refinement, orbit the model, download the GLB and open it independently. Present measured time, resource use and reproducibility instructions. If demonstrating CPU mode, identify it explicitly as a geometric baseline. A polished UI and CPU output alone do not prove the generative AI portion of the brief.

Sources and attribution

The requirement assessment is based on the supplied TechZephyr 4.0 Sketch2Shape AI problem statement and the delivered source code, tests and local sample report. No judging score or event acceptance is guaranteed.

TripoSR official source and setup: https://github.com/VAST-AI-Research/TripoSR

ControlNet pipeline documentation: https://huggingface.co/docs/diffusers/en/api/pipelines/controlnet

ControlNet model: https://huggingface.co/lllyasviel/control_v11p_sd15_scribble

Stable Diffusion model: https://huggingface.co/stable-diffusion-v1-5/stable-diffusion-v1-5

Google model-viewer source and license: https://github.com/google/model-viewer

Mesh library documentation: https://trimesh.org/

Original application code uses the included MIT license. Third-party libraries and pretrained weights retain their own licenses and model terms. TripoSR and viewer attribution is included in the README, and the viewer license is bundled. This project integrates pretrained models; it does not claim authorship of those models.